

Sprints - 3

EV Challenge - GoodWe



Integrantes:

Caio Henrique Ferraz da Silva (568992)

Enzo Caruso Peter (570908)

Leonardo Figueredo dos Santos (573653)

Leonardo Robert Maulicino (570329)

Lucas Ramos de Sousa (573901)

Matheus Pimenta Martini (569400)

Pablo Renato dos Santos Sobral de Carvalho (569894)

1. Resumo da Evolução

“O que existia nas Sprints 1/2 → o que existe agora na Sprint 03”

a) Memória persistida manualmente → Token Buffer Memory

Sprint 1/2: histórico como lista de dicionários (user/assistant), truncado por número máximo de mensagens, com log interno separado.

Sprint 3: a memória é limitada por tokens, sendo configurável e administrada pelo LangChain usando *ConversationTokenBufferMemory*.

b) Invocação manual de funções → Pipelines declarativas (LCEL)

Sprint 1/2: chamadas sequenciais manuais na classe *ChargeGridChatbot*, código verboso.

Sprint 3: declaração do fluxo (entidades e dados) com LCEL e o operador “|”, execução (síncrona, assíncrona, streaming, batch) delegada ao *framework*. Arquivo *chatbot.py* se tornou obsoleto, enquanto o *app.py*, usando os recursos LangChain construídos no *builder*, ficou responsável pelo gerenciamento do fluxo conversacional.

c) Output não estruturado/validado → Output com schema Pydantic

Sprint 1/2: resposta como string livre, sem validação, sem retry; risco de dados incorretos e vazamento.

Sprint 3: validação por BaseModel + PydanticOutputParser, campos opcionais/padrões. Usado no chatbot para retornar um relatório semanal de recargas.

d) System Prompt em Markdown → System Prompt com tags XML e organização estratégica

Sprint 1/2: blocos em Markdown, seções de identidade/persona/contexto/guardrails.

Sprint 3: delimitação por tags XML (reduz ambiguidade) e seções críticas no início/fim para melhorar a atenção do modelo.

e) Guardrail restringido ao *system prompt* → Camadas *scope_validator* e *moderation*

Sprint 1/2: a única camada que protege a LLM de prompts maliciosos era o *system prompt*.

Sprint 3: guardrails baseados em regras e expressões regulares. Normalizam as entradas, identificam padrões de *jailbreak*, *prompt injection*, solicitações fora do escopo ou de informações inexistentes na base, bloqueiam essas solicitações com mensagens específicas e, após a geração da resposta, verificam possíveis vazamentos de informações internas

2. Refatoração LangChain

a) *Token Buffer Memory*

Após a refatoração, o chatbot possui uma memória baseada em número definido de tokens, permitindo controle mais preciso de recursos. Ademais, ela é facilmente configurável e administrada pelo *LangChain* através da sua linguagem declarativa, o que torna o código muito mais enxuto.

b) *Pipelines declarativas (LCEL)*

Agora, apenas cabe ao desenvolvedor declarar o fluxo do código (entidades e fluxo de dados); a execução (síncrona, assíncrona, streaming, batch) fica com o *framework*. Isso é possível graças ao LCEL, que usa o operador “|” para encadear esses componentes de forma legível .

c) *Output com schema Pydantic e retry*

Na versão refatorada do *chatbot*, a saída do relatório de recargas é validada segundo um *schema* Pydantic, garantindo que cada *field* da resposta do modelo siga um formato e regras definidas, como campos opcionais e valores padrões. Nos casos em que o retorno não seja válido, é possível embutir futuramente um mecanismo de *retry* interno, que pede automaticamente ao modelo para reformatar a saída conforme o *schema*.

d) *System Prompt* com tags XML e organização estratégica

Os blocos do *system prompt* agora estão delimitados por tags XML, reduzindo possíveis ambiguidades do uso de Markdowns e auxiliando a LLM a priorizar seções do contexto. Além disso, com novas versões do *system prompt*, seções mais críticas foram colocadas no começo e no fim da estrutura para que não fossem sub-representadas pelos mecanismos de atenção do modelo. Técnica *few-shot* mantida.

3. Tabela Comparativa

Métricas	Sprints 1/2 (versão manual/legado)	Sprint 3 (LCEL)
Nota no Eval	10	10
Tokens por turno	967 (média)	≈ 1340
Latência média	≈1,723s	≈3,334s
Acurácia Structured Output	Não implementado	100% (teste 6)

Na versão legado, os tokens contabilizados incluem contexto + pergunta, enquanto a versão refatorada, além destes, considera histórico (corroborando para aumento de tokens por turno), com *TokenBufferMemory* com 1100 tokens de limite. Aumento de latência devido à execução da *chain* de classificação (não presente na versão legado).

Observação: 3 testes foram adicionados à nova bateria de testes para contemplarem os requisitos desta *Sprint*. Eles não possuem baseline correspondente na versão legado e, portanto, não entram no cálculo da comparação quantitativa antes × depois.

4. Problemas encontrados e soluções

PROBLEMA 1: A LLM perde o contexto do primeiro turno de conversa após poucos turnos dependendo do volume de texto gerado.

SOLUÇÃO: Aumentar o número de tokens no *TokenBufferMemory* para 2300 fez com que o modelo preservasse mais contexto da conversa.

PROBLEMA 2: O *system prompt* v1 do *router* às vezes direciona a entrada "Alguma anomalia foi registrada essa semana?" para a chain estruturada (devia ser *nao_estruturada*). Além disso, já retornou a resposta correta, mas fora do *schema* e já retornou a resposta estruturada que não correspondia à pergunta.

SOLUÇÃO: Refinamos o *system prompt*, adicionando blocos demarcados por tags XML, regras de classificação, exemplos de entrada que deviam receber saída validada pelo Pydantic. Depois disso, a acurácia de classificação da entrada do usuário aumentou consideravelmente, além de evitar que o modelo retornasse saídas incompatíveis com o *schema* Pydantic.

5. Equipe e divisão de trabalho

Os integrantes diretamente envolvidos no projeto e suas responsabilidades foram:

Pablo Renato: composição das *chains* de classificação, conversacional e de saída estruturada, *schemas* Pydantic, memória conversacional, versionamento de *system prompts*, reexecução e documentação da bateria de testes na versão refatorada e *relatorio_modelos.md*.

Leonardo Figueredo: criação dos *guardrails* *scope_validator* e *moderation*.

Lucas Ramos: integração das *pipelines* *LangChain* desenvolvidas à versão legado e à interface *streamlit*, integração dos *guardrails* ao *chatbot* e documentação README.